VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

OPERATING SYSTEMS (CO2017)

Assignment

"Simple Operating System"

Instructor(s): Nguyễn Quang Hùng

Lab: L04

Ho Chi Minh City, April 27, 2025

Mục lục

Danh sách thành viên **2**

1 Scheduler **3**

1.1 Trả lời câu hỏi 3

1.2 Phân tích 3

1.3 Testcase và biểu đồ Gantt 6

2 Memory Management **9**

2.1 Trả lời câu hỏi 10

2.2 Libmem 12

2.3 mm-vm 15

2.4 mm 16

2.5 memphy 19

2.6 Testcase os_1_mlq_paging 20

3 System Call **22**

3.1 Trả lời câu hỏi 22

3.2 Sự tương tác giữa các module khi gọi System Call 23

3.3 Triển khai System Call killall 24

Danh sách thành viên

STT	Họ tên	MSSV	CPU Scheduling	Memory	System Call
1	Võ Công Đăng Khôi	2311708			✓
2	Nguyễn Thị Cẩm Vân	2313870	✓		
3	Huỳnh Kim Quý	2312894		✓	
4	Hồng Phi Trường	2313703		✓	

1 Scheduler

1.1 Trả lời câu hỏi

Question: What are the advantages of using the scheduling algorithm described in this assignment compared to other scheduling algorithms you have learned?

Chiến lược định thời được sử dụng là hàng chờ đa cấp dựa trên độ ưu tiên (multilevel queue).

Ưu điểm:

- **Đảm bảo tiến trình quan trọng được xử lý trước:** Căn cứ vào độ ưu tiên giúp các tiến trình quan trọng được thực thi sớm, rút ngắn thời gian hoàn thành công việc.
- **Giảm thời gian chờ trung bình:** Các tiến trình có nhu cầu CPU lớn sẽ được phân phối hợp lý hơn, giúp cải thiện hiệu suất hệ thống.
- **Ngăn chặn tiến trình ưu tiên thấp bị "đói CPU":** Cơ chế slot đảm bảo sau khi các hàng đợi ưu tiên cao dùng hết lượt của mình, hàng đợi thấp vẫn được cấp CPU.
- **Phù hợp với môi trường thời gian thực:** Các ứng dụng có yêu cầu tài nguyên và thời gian thay đổi liên tục sẽ được đáp ứng tốt hơn nhờ cơ chế phân loại tiến trình theo mức ưu tiên.

1.2 Phân tích

1.2.1 queue.c

Các hàm chính:

```c
void enqueue(struct queue_t * q, struct pcb_t * proc) {
        /* TODO: put a new process to queue [q] */

        if (q->size < MAX_QUEUE_SIZE){
                for (int i = 0; i < q->size; i++) {
                    if (q->proc[i]->pid == proc->pid){
                        return;
                    }
                }
        q->proc[q->size] = proc;
        q->size++;
    }
}
```

Hàm enqueue có nhiệm vụ thêm một tiến trình mới vào cuối hàng đợi. Trước tiên, hàm kiểm tra xem số lượng tiến trình hiện có (size) trong hàng đợi có nhỏ hơn giới hạn tối đa (MAX_QUEUE _SIZE) hay không. Nếu còn chỗ, tiến hành kiểm tra xem tiến trình cần thêm đã tồn tại trong hàng đợi chưa. Nếu tiến trình chưa có mặt trong hàng đợi, hàm sẽ thêm nó vào vị trí cuối cùng (chỉ mục size) và tăng biến size của hàng đợi lên một đơn vị.

```
struct pcb_t * dequeue(struct queue_t * q) {
        /* TODO: return a pcb whose prioprity is the highest
         * in the queue [q] and remember to remove it from q
         * */
        if(empty(q)) return NULL;
        struct pcb_t*proc = q->proc[0];
        for(int i = 0; i < q->size - 1; i++){
                q->proc[i] = q->proc[i+1];
        }
        q->proc[q->size - 1] = NULL;
        q->size--;
        return proc;
}
```

Hàm dequeue được sử dụng để loại bỏ và trả về tiến trình đầu tiên trong hàng đợi. Đầu tiên kiểm tra hàng đợi có trống hay không. Nếu hàng đợi trống trả về NULL. Ngược lại, theo thuật toán Multi-Level Queue (MLQ), các tiến trình có cùng mức ưu tiên (prio) được gom vào cùng một hàng đợi riêng biệt. Do đó, hàm dequeue sẽ loại bỏ phần tử ở vị trí đầu tiên (chỉ mục 0) của hàng đợi và trả về phần tử đó. Sau khi lấy tiến trình này, toàn bộ các phần tử còn lại trong hàng đợi sẽ được dịch lên trước một vị trí để lấp khoảng trống. Phần tử cuối cùng (chỉ mục size - 1) được đặt thành NULL, sau đó giảm size của hàng đợi đi một đơn vị.

1.2.2 sched.c

Các hàm chính:

```
void put_mlq_proc(struct pcb_t * proc) {
    pthread_mutex_lock(&queue_lock);
    enqueue(&mlq_ready_queue[proc->prio], proc);
    pthread_mutex_unlock(&queue_lock);
}

void put_proc(struct pcb_t * proc) {
    proc->ready_queue = &ready_queue;
    proc->mlq_ready_queue = mlq_ready_queue;
```

```
    proc->running_list = & running_list;
    /* TODO: put running proc to running_list */

    pthread_mutex_lock(&queue_lock);
    enqueue(&running_list, proc);
    pthread_mutex_unlock(&queue_lock);
    return put_mlq_proc(proc);
}
```

Hàm put_proc được gọi khi tiến trình bị ngắt hoặc nhường CPU. Tiến trình được đưa vào danh sách running_list để ghi nhận nó đã từng chạy và sau đó đưa tiến trình vào mlq_ready_queue phù hợp với mức ưu tiên. Tác dụng đưa một tiến trình đang thực thi trở lại hàng đợi nếu nó chưa hoàn thành công việc.

```
void add_mlq_proc(struct pcb_t * proc) {
    pthread_mutex_lock(&queue_lock);
    enqueue(&mlq_ready_queue[proc->prio], proc);
    pthread_mutex_unlock(&queue_lock);
}

void add_proc(struct pcb_t * proc) {
    proc->ready_queue = &ready_queue;
    proc->mlq_ready_queue = mlq_ready_queue;
    proc->running_list = & running_list;
    /* TODO: put running proc to running_list */

    pthread_mutex_lock(&queue_lock);
    enqueue(&running_list, proc);
    pthread_mutex_unlock(&queue_lock);
    return add_mlq_proc(proc);
}
```

Hàm add_proc thêm một tiến trình mới hoặc tiến trình sẵn sàng trở vào lại hàng đợi sẵn sàng để chuẩn bị được CPU xử lý. Được gọi khi hệ thống tạo một tiến trình mới hay khi một tiến trình đã hoàn thành I/O và quay lại trạng thái sẵn sàng. Tiến trình được đưa vào hàng đợi mlq_ready_queue và đồng thời đưa vào running_list để ghi nhận nó đã từng chạy hoặc chờ chạy.

```
struct pcb_t * get_mlq_proc(void) {
    pthread_mutex_lock(&queue_lock);
    struct pcb_t *proc = NULL;
    for (int prio = 0; prio < MAX_PRIO; prio++) {
```

```
        if (!empty(&mlq_ready_queue[prio])) {
            if (slot[prio] < (MAX_PRIO - prio)) {
                proc = dequeue(&mlq_ready_queue[prio]);
                slot[prio]++;
                pthread_mutex_unlock(&queue_lock);
                return proc;
            }
        }
    }

    for (int i = 0; i < MAX_PRIO; i++) {
        slot[i] = 0;
    }
    pthread_mutex_unlock(&queue_lock);
    return NULL;
}

struct pcb_t * get_proc(void) {
    return get_mlq_proc();
}
```

Hàm get_proc lấy tiến trình sẵn sàng tiếp theo để cấp CPU thực thi. Đầu tiên, hàm duyệt qua các hàng đợi trong mlq_ready_queue theo thứ tự độ ưu tiên từ cao đến thấp. Với mỗi hàng đợi, nếu hàng đợi không rỗng và số lần cấp CPU cho hàng đợi này (biến slot) chưa vượt quá giới hạn cho phép (MAX_PRIO - prio), thì một tiến trình sẽ được dequeue từ hàng đợi đó, đồng thời slot[prio] được tăng lên 1 đơn vị và trả về tiến trình đó.

Nếu không chọn được tiến trình nào sau khi duyệt toàn bộ các hàng đợi, tức là tất cả đã vượt quá giới hạn cấp CPU, thì hàm sẽ reset lại toàn bộ mảng slot[] để bắt đầu một lượt cấp phát mới, đảm bảo công bằng giữa các mức ưu tiên theo cơ chế vòng quay có trọng số.

1.3 Testcase và biểu đồ Gantt

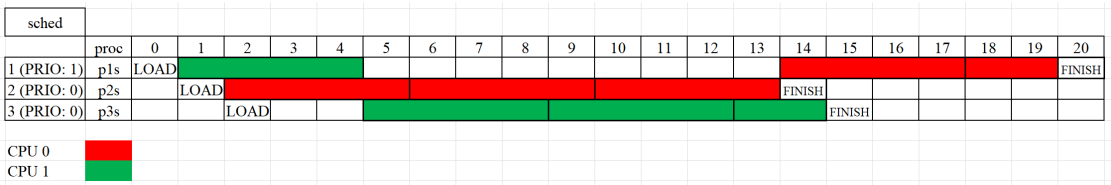
1.3.1 Test sched

Input:

```
4 2 3
0 p1s 1
1 p2s 0
2 p3s 0
```

Giải thích: Time slice = 4, số CPU = 2, số process = 3. Tại thời điểm 0, tiến trình p1s (có độ ưu tiên mặc định là 1 và 10 lệnh CALC) được nạp vào và ghi đè độ ưu tiên mặc định trong file p1s thành 1. Tại thời điểm 1, tiến trình p2s (có độ ưu tiên mặc định là 20 và 12 lệnh calc) được nạp vào và ghi đè độ ưu tiên khi chạy là 0. Tại thời điểm 2, tiến trình p3s (độ ưu tiên mặc định là 7 và 11 dòng calc) được nạp vào và ghi đè độ ưu tiên khi chạy là 0.

Gantt chart:



Giải thích:

Tại time slot 0, tiến trình p1s (PID 1, priority 1) được nạp. Đến slot 1, p1s được CPU 1 thực thi, trong khi tiến trình p2s (PID 2, priority 0) được nạp.

Do p2s có độ ưu tiên cao hơn, CPU 0 bắt đầu chạy p2s ở slot 2, cùng lúc tiến trình p3s (PID 3, priority 0) được nạp. Sau đó, các CPU luân phiên chạy p1s, p2s, và p3s theo Round Robin, mỗi tiến trình chạy 2 slot.

Khi p2s hoàn tất ở slot 14, CPU 0 quay lại chạy p1s. Sau đó, p3s cũng hoàn thành ở slot 15, khiến CPU 1 dừng tại slot 16. Cuối cùng, p1s tiếp tục chạy và hoàn tất ở slot 20, rồi CPU 0 cũng dừng lại.

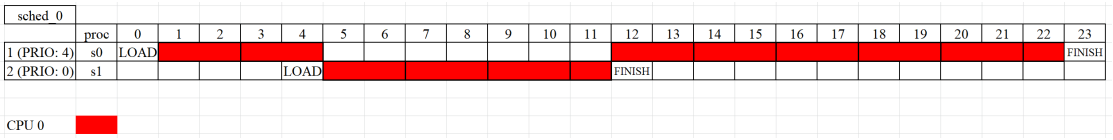
1.3.2 Test sched_0

Input:

```
2 1 2
0 s4 4
4 s1 0
```

Giải thích: Time slice = 2, số CPU = 1, số process = 2. Tại thời điểm 0, tiến trình s0 (có độ ưu tiên mặc định là 12 và 15 lệnh CALC) được nạp vào và ghi đè độ ưu tiên khi chạy là 4. Tại thời điểm 4, tiến trình s1 (có độ ưu tiên mặc định là 20 và 7 lệnh calc) được nạp vào và ghi đè độ ưu tiên khi chạy là 0.

Gantt chart:



Giải thích:

Khi chương trình bắt đầu tại thời điểm 0, tiến trình s0 (PID 1, priority 4) được nạp và bắt đầu chạy từ slot 1. Đến slot 4, sau khi hết time slice, tiến trình này bị ngắt. Đúng lúc đó, tiến trình s1 (PID 2, priority 0) được nạp với độ ưu tiên cao hơn, nên CPU chuyển sang chạy s1 từ slot 5. Sau các lượt chạy xen kẽ, s1 hoàn tất tại slot 12. CPU quay lại tiếp tục tiến trình s0 từ slot 13. Vì không còn tiến trình nào khác, s0 được thực thi liên tục và hoàn thành tại slot 23. CPU dừng vì không còn tiến trình nào đang chạy.

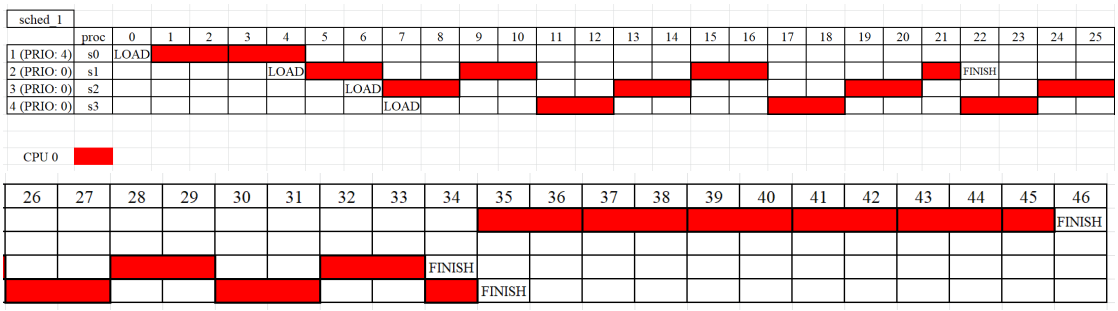
1.3.3 Test sched_1

Input:

```
2 1 4
0 s0 4
4 s1 0
6 s2 0
7 s3 0
```

Giải thích: Time slice = 2, số CPU = 1, số process = 4. Tại thời điểm 0, tiến trình s0 (có độ ưu tiên mặc định là 12 và 15 lệnh CALC) được nạp vào và ghi đè độ ưu tiên khi chạy là 4. Tại thời điểm 4, tiến trình s1 (có độ ưu tiên mặc định là 20 và 7 lệnh calc) được nạp vào và ghi đè độ ưu tiên khi chạy là 0. Tại thời điểm 6, tiến trình s2 (độ ưu tiên mặc định là 20 và 12 dòng calc) được nạp vào và ghi đè độ ưu tiên khi chạy là 0. Tại thời điểm 7, tiến trình s3 (độ ưu tiên mặc định là 7 và 11 dòng calc) được nạp vào và ghi đè độ ưu tiên khi chạy là 0.

Gantt chart:



Giải thích:

Ở time slot 0, tiến trình s0 (PID 1, PRIO 4) được nạp vào hệ thống. CPU 0 bắt đầu thực thi tiến trình này từ slot 1. Sau 3 slot, đến slot 4, tiến trình s1 (PID 2, PRIO 0) được nạp, có ưu tiên cao hơn nên ngay slot 5, s0 bị ngắt và CPU chuyển sang chạy s1.

Tiếp theo, lần lượt các tiến trình s2 (PID 3) và s3 (PID 4) cũng được nạp vào ở slot 6 và 7, cả hai đều có PRIO 0. Từ đây, CPU thực hiện luân phiên giữa các tiến trình s1, s2, s3, s4 theo vòng tròn (Round Robin), mỗi tiến trình được chạy 2 slot thì nhường lượt.

Sau khi tiến trình s1 hoàn thành ở slot 22, ba tiến trình còn lại tiếp tục xoay vòng cho đến khi s3 kết thúc ở slot 34 và s4 kết thúc ở slot 35.

Cuối cùng, CPU quay lại thực hiện tiến trình s0 (PRIO thấp hơn), tiếp tục cho đến khi tiến trình này hoàn tất vào slot 46. Khi không còn tiến trình nào nữa, CPU dừng hoạt động.

2 Memory Management

Trong Simple OS, bộ nhớ được quản lý dựa trên mô hình phân trang (paging) kết hợp với ý tưởng phân vùng (region) và vùng nhớ ảo riêng biệt cho từng tiến trình.

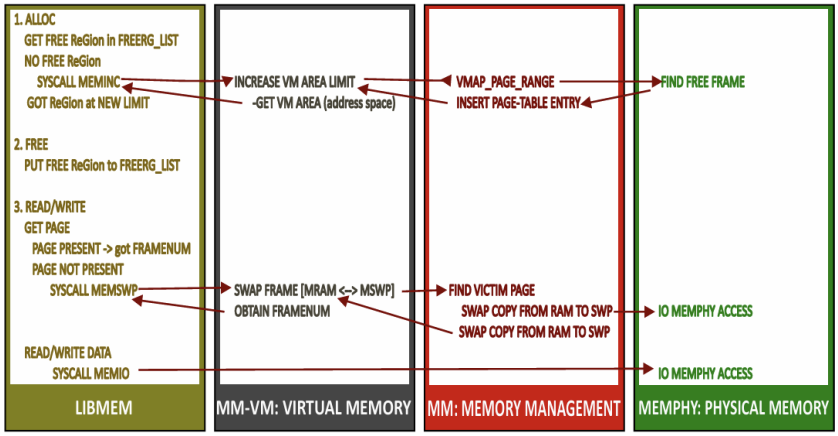
Mỗi tiến trình được cấp phát một không gian bộ nhớ ảo, bao gồm nhiều vùng nhớ (vm_area_struct), tương ứng với các segment như heap, data,... Trong mỗi vùng, khi chương trình yêu cầu cấp phát (ALLOC), hệ điều hành sẽ: Xác định một region tương ứng trong vùng đó, tìm kiếm và cấp phát các frame từ bộ nhớ vật lý (RAM) và ánh xạ địa chỉ ảo của tiến trình sang địa chỉ vật lý thông qua bảng trang (page table). Hệ thống sử dụng địa chỉ ảo 22-bit, với kích thước mỗi frame là 256 byte, tương đương với 16.384 trang (pages) trong không gian ảo. Mỗi entry trong page table là một từ 32-bit chứa thông tin như Frame number (nếu page đang ở RAM), Bit "present" cho biết page có đang ở RAM hay không, Bit "swapped" nếu page đã được chuyển sang SWAP Hình dưới đây là sơ đồ luồng xử lý quản lý bộ nhớ trong hệ mô phỏng và các src cần hiện thực:

LIBMEM: Thư viện cung cấp các syscall như MEMINC, MEMSWP, MEMPIO, tương tác với hệ thống quản lý bộ nhớ.

MM-VM: Quản lý bộ nhớ ảo, bao gồm các vùng địa chỉ (vm_area), ánh xạ page table, xử lý giới hạn bộ nhớ ảo.

MM (Memory Management): Lõi xử lý cấp phát khung trang (frame), swap giữa RAM và SWAP, tìm victim page.

MEMPHY: Truy cập bộ nhớ vật lý, xử lý IO thực tế đến RAM hoặc SWAP.



Hình 1: Memory system modules

2.1 Trả lời câu hỏi

Question: In this simple OS, we implement a design of multiple memory segments or memory areas in source code declaration. What is the advantage of the proposed design of multiple segments?

Hệ điều hành mô phỏng này bố trí bộ nhớ ảo của một tiến trình theo từng vùng (segment), cụ thể trong cấu trúc `vm_area_struct`, mỗi vùng bộ nhớ `vm_area` có:

- `vm_start`: địa chỉ bắt đầu.
- `vm_end`: địa chỉ kết thúc.
- `sbrk`: con trỏ quản lý heap.
- Một danh sách các vùng trống (freerg_list).

Mỗi process có thể có nhiều `vm_area_struct` được quản lý bởi `mm_struct` → góp phần phân vùng bộ nhớ thành code, heap và stack (mặc dù phần code không đặt trong RAM, đơn giản hóa cho việc mô phỏng).

Ưu điểm của thiết kế nhiều vùng:

- Phân chia hợp lý các phân đoạn: STACK, HEAP, CODE, DATA → dễ bảo vệ và quản lý.
- Có thể cấp phát/thu hồi bộ nhớ độc lập trong từng `vm_area` → dễ tránh hiện tượng fragmentation.
- Tạo tiền đề để áp dụng kỹ thuật bảo mật và phân phối quyền truy cập từng vùng trong thực tế.

- Hỗ trợ mở rộng flexible heap thông qua `sbrk()` và `vm_freerg_list`.

Question: What will happen if we divide the address to more than 2 levels in the paging memory management system?

Nếu ta chia hệ thống quản lý địa chỉ thành phân trang nhiều cấp (multi-level paging), ví dụ: 2 cấp → mỗi bảng là con trỏ đến bảng phụ, thay vì bảng trang lớn duy nhất.

Hệ quả của chia nhiều cấp:

Ưu điểm:

- Tiết kiệm không gian bộ nhớ cho page table → chỉ tạo khi cần → giảm overhead khi số trang nhỏ.
- Dễ mở rộng hệ thống cho address space rất lớn (ví dụ: 64-bit).
- Ít tốn bộ nhớ vật lý cho bảng trang khi tiến trình dùng ít ram.

Nhược điểm:

- Làm chậm quá trình dịch địa chỉ (address translation) vì phải truy xuất từng cấp bảng.
- Gây khó khăn khi lập trình mô phỏng vì phải quản lý nhiều cấu trúc bảng liên kết.

Question: What are the advantages and disadvantages of segmentation with paging?

Trong hệ điều hành mô phỏng, mỗi tiến trình có thể có nhiều phân đoạn (heap, data...) và trong mỗi phân đoạn sẽ phân trang bộ nhớ - tức là sử dụng mô hình "segmentation + paging".

Ưu điểm:

- Cho phép quản lý bộ nhớ hiệu quả nhờ chia nhỏ logical space (segmentation) kết hợp quản lý vật lý theo frame (paging).
- Hạn chế fragmentation bên trong (internal fragmentation).
- Tăng bảo mật: mỗi segment có thể đặt quyền riêng biệt (read-only code, writable data).
- Dễ mở rộng phân vùng riêng biệt (ví dụ: `sbrk()` mở rộng vùng heap).

Nhược điểm:

- Cần quản lý đồng thời cấu trúc segment + page → phức tạp hơn về cài đặt so với chỉ phân trang.
- Tăng overhead trong dịch địa chỉ (cần tra `vm_area` → `page` → `frame`). Khó khăn trong bảo trì hệ thống lớn nếu không có công cụ trợ giúp (MMU, TLB...).

2.2 Libmem

libmem.c dùng để quản lý bộ nhớ ảo theo cơ chế phân trang (paging) trong một mô phỏng hệ điều hành, nó có các phương thức quan trọng nhu alloc, write, read,...

2.2.1 __*alloc*

Chức năng: Cấp phát một vùng nhớ (region) cho tiến trình.

```
int __alloc(struct pcb_t *caller, int vmaid, int rgid, int size, int
    *alloc_addr)
{
 struct vm_rg_struct rgnode;
 if (get_free_vmrg_area(caller, vmaid, size, &rgnode) == 0) {
   caller->mm->symrgtbl[rgid].rg_start = rgnode.rg_start;
   caller->mm->symrgtbl[rgid].rg_end = rgnode.rg_end;
   *alloc_addr = rgnode.rg_start;
   return 0;
 }
 /*Attempt to increate limit to get space */
 struct vm_area_struct *cur_vma = get_vma_by_num(caller->mm, vmaid);
 int inc_sz = PAGING_PAGE_ALIGNSZ(size);
 int old_sbrk ;
 old_sbrk = cur_vma->sbrk;
 printf("old_sbrk: %d\n", old_sbrk);
 /* TODO INCREASE THE LIMIT
  * inc_vma_limit(caller, vmaid, inc_sz)
  */
 if(!inc_vma_limit(caller, vmaid, inc_sz)){
   if(inc_sz > size){
     struct vm_rg_struct *rgnode = malloc(sizeof(struct vm_rg_struct)
         );
     rgnode->rg_start = size + old_sbrk + 1;
     rgnode->rg_end = inc_sz + old_sbrk;
     enlist_vm_freerg_list(caller->mm, rgnode);
   }
   cur_vma->sbrk += inc_sz;
   /*Successful increase limit */
   caller->mm->symrgtbl[rgid].rg_start = old_sbrk;
   caller->mm->symrgtbl[rgid].rg_end = old_sbrk + size;
   *alloc_addr = old_sbrk;
```

```
    printf("alloc_addr: %d\n", *alloc_addr);
    printf("PID=%d - Region=%d - Address=%08x - Size=%d byte\n",
        caller->pid, rgid, *alloc_addr, size);
  }
  return 0;
}
```

Giải thích: Hàm alloc ban đầu kiểm tra xem trong danh sách vùng nhớ trống hay không, nếu có thì dùng nó để cấp phát rồi return 0. Nếu không, tăng giới hạn vùng nhớ, ghi địa chỉ cũ sbrk để cấp phát. Nếu có phần thừa sau khi mở rộng thì đưa vào danh sách trống và cập nhật thông tin vào bảng ký hiệu.

2.2.2 $__free(...)$

Chức năng: Giải phóng vùng nhớ đã cấp phát.

```
int __free(struct pcb_t *caller, int vmaid, int rgid)
{
  struct vm_rg_struct rgnode;
  if (rgid < 0 || rgid > PAGING_MAX_SYMTBL_SZ) return -1;
  /* TODO: Manage the collect freed region to freerg_list */
        rgnode = *get_symrg_byid(caller->mm, rgid);
    enlist_vm_freerg_list(caller->mm, &rgnode);
    caller->mm->symrgtbl[rgid].rg_start = 0;
    caller->mm->symrgtbl[rgid].rg_end = 0;
  return 0;
}
```

Giải thích: free Kiểm tra chỉ số rgid có nằm trong giới hạn hợp lệ hay không, nếu có thì trả về -1. nếu không thì gọi get_symrg_byid trả về con trỏ đến vùng nhớ (region) có rgid trong bảng ký hiệu symrgtbl gán vào biến rgnode. Gọi enlist_vm_freerg_list để thêm vùng nhớ trên vào danh sách trống, cuối cùng là xóa địa chỉ bắt đầu và kết thúc của vùng nhớ đó.

2.2.3 *pg_getpage*

Chức năng: Đảm bảo rằng trang (page) có số pgn (page number) đã được load vào RAM. Nếu trang này chưa có trong RAM (tức nằm trên swap), thì sẽ thực hiện hoán đổi (swap) để mang nó từ swap space vào RAM và đẩy một trang khác ra ngoài nếu cần.

```
int pg_getpage(struct mm_struct *mm, int pgn, int *fpn, struct pcb_t *
    caller)
```

```
{
  uint32_t pte = mm->pgd[pgn];
  if (!PAGING_PAGE_PRESENT(pte))
  { /* Page is not online, make it actively living */
    int vicpgn, swpfpn, vicfpn;
    uint32_t vicpte;
    int tgtfpn = PAGING_PTE_SWP(pte);
    /* Find a victim page to evict from RAM */
    find_victim_page(caller->mm, &vicpgn);
    if (vicpgn == -1) return -1;
    vicpte = mm->pgd[vicpgn];
    vicfpn = PAGING_PTE_PGN(vicpte);
    /* Get free frame in MEMSWP */
    MEMPHY_get_freefp(caller->active_mswp, &swpfpn);
    if (swpfpn == -1)
        return -1;
    /* Do swap frame from MEMRAM to MEMSWP and vice versa */
    /* Copy victim frame to swap */
    __swap_cp_page(caller->mram, vicfpn, *(caller->mswp), swpfpn);
    /* Copy target frame from swap to mem */
    __swap_cp_page(*(caller->mswp), tgtfpn, caller->mram, vicfpn);
    /* Update page table */
    pte_set_swap(&mm->pgd[vicpgn], 1, PAGING_OFFST(vicpgn));
    /* Update its online status of the target page */
    pte_set_fpn(&mm->pgd[pgn], tgtfpn);
    enlist_pgn_node(&caller->mm->fifo_pgn, pgn);
  }
  *fpn = PAGING_FPN(mm->pgd[pgn]);
  return 0;
}
```

Giải thích: Hàm này ban đầu truy cập vào bảng trang, lấy thông tin của trang pgn. Nếu trang chưa có mặt trong RAM, tìm một trang vicpgn đang nằm trong RAM để thay thế. Tìm một frame trống trong vùng swap sau đó hoán đổi dữ liệu sau đó cập nhật lại bảng trang và gáng lại con trỏ fpn.

2.2.4 __*read*(...)

```
int __read(struct pcb_t *caller, int vmaid, int rgid, int offset, BYTE
    *data)
{
```

```
  struct vm_rg_struct *currg = get_symrg_byid(caller->mm, rgid);
  struct vm_area_struct *cur_vma = get_vma_by_num(caller->mm, vmaid);
  if (currg == NULL || cur_vma == NULL) /* Invalid memory identify */
    return -1;
  pg_getval(caller->mm, currg->rg_start + offset, data, caller);
  return 0;
}
```

Chức năng: Đọc 1 byte từ vùng nhớ ảo đã cấp phát: Lấy thông tin vùng nhớ thông qua rgid qua currg. Xác định vùng nhớ ảo (VMA) mà vùng rgid thuộc về. Nếu tìm thấy thì gọi pg_getval để đọc giá trị vào *data, nếu không thì trả về -1.

2.2.5 __*write*(...)

```
int __write(struct pcb_t *caller, int vmaid, int rgid, int offset,
    BYTE value)
{
  struct vm_rg_struct *currg = get_symrg_byid(caller->mm, rgid);
  struct vm_area_struct *cur_vma = get_vma_by_num(caller->mm, vmaid);
  if (currg == NULL || cur_vma == NULL) return -1;
  pg_setval(caller->mm, currg->rg_start + offset, value, caller);
  return 0;
}
```

Hàm này tương tự như read nhưng gọi pg_setval thay vì getval.

2.3 mm-vm

Quản lý bộ nhớ ảo, bao gồm các vùng địa chỉ, ánh xạ page table, xử lý giới hạn bộ nhớ ảo. Dưới đây là cấu trúc bộ nhớ ảo trong mô phỏng.

```
/*
mm_struct (bộ nhớ của tiến trình)
|
├── mmap (Danh sách vùng nhớ ảo - VMA)
|   ├── vm_area_struct (Vùng nhớ 1)
|   |   ├── vm_freerg_list (Danh sách vùng trống trong vùng nhớ này)
|   |   |   ├── vm_rg_struct (Vùng trống 1)
|   |   |   ├── vm_rg_struct (Vùng trống 2)
|   |   |   └── ...
|   |   └── vm_next → vm_area_struct (Vùng nhớ 2)
|   ├── vm_area_struct (Vùng nhớ 2)
|   └── ...
|
├── symrgtbl[] (Bảng biểu tượng: thông tin các vùng đã cấp phát theo biến/rgid)
|   ├── vm_rg_struct (vùng của biến reg0)
|   ├── vm_rg_struct (vùng của biến reg1)
|   └── ...
|
├── pgd[] (Page Table: ánh xạ page number → frame number trong RAM)
|   ├── PTE[0] → Frame 1
|   ├── PTE[1] → Frame 4
|   └── ...
|
└── fifo_pgn (Danh sách các page đã dùng gần nhất – phục vụ page replacement FIFO)

*/
```

Hình 2: Cấu trúc bộ nhớ

Dưới đây là một số hàm cơ bản:

Hàm	**Chức năng**
get_vma_by_num(...)	Duyệt qua danh sách liên kết mmap và trả về `vm_area_struct` có `vm_id = vmaid`.
__mm_swap_page(...)	Hoán trang giữa RAM và swap, gọi hàm `__swap_cp_page()` để copy dữ liệu từ RAM sang SWAP.
get_vm_area_node_at_brk(...)	Tạo node `vm_rg_struct` mới tại cuối VMA (sử dụng `cur_vma->sbrk` và `alignedsz`).
validate_overlap_vm_area(...)	Kiểm tra xem vùng cấp phát mới có overlap với vùng khác không.
inc_vma_limit(...)	Tăng giới hạn vùng nhớ ảo để cấp phát thêm trang mới cho tiến trình.

2.4 mm

mm.c là lõi xử lý cấp phát khung trang (frame), swap giữa RAM và SWAP, tìm victim page. Một số hàm và chức năng cụ thể:

init_pte, pte_set_fpn, pte_set_swap: Quản lý và cập nhật entry trong Page Table.

alloc_pages_range: cấp phát req_pgnum khung trang vật lý

vmap_page_range: ánh xạ nhiều trang vào bảng trang

vm_map_ram: kết hợp hai hàm trên để cấp phát RAM và ánh xạ vào địa chỉ ảo

__swap_cp_page: Thực hiện copy nội dung trang từ RAM và Swap

init_mm, init_vm_rg, enlist_vm_rg_node: Khởi tạo hệ thống quản lý bộ nhớ: gán vm_start, sbrk, vm_end cho vm_area. Tạo danh sách vùng trống ban đầu (freerg_list). Cấp phát bảng trang pgd[]. Mỗi tiến trình khi khởi tạo sẽ có một vùng nhớ ảo mặc định, khởi đầu rỗng, nhưng có vùng trống freerg_list để sẵn sàng cấp phát.

2.4.1 vmap_page_range(...)

```c
int vmap_page_range(struct pcb_t *caller,
                    int addr,
                    int pgnum,
                    struct framephy_struct *frames,
                    struct vm_rg_struct *ret_rg)
 {
   int pgit = 0;
   int pgn = PAGING_PGN(addr);
   // TODO: update the rg_end and rg_start of ret_rg
   ret_rg->rg_end = (addr_t)(addr + pgnum * PAGING_PAGESZ);
   ret_rg->rg_start = (addr_t)(addr);
   /* TODO map range of frame to address space
    *      [addr to addr + pgnum*PAGING_PAGESZ
    *      in page table caller->mm->pgd[]
    */
   for (int i=0; i < pgnum && frames != NULL;i++) {
     caller->mm->pgd[pgn + i] = (frames->fpn << OFFSET_LEN | 0
         x80000000);
     frames = frames->fp_next;
   }
   /* Tracking for later page replacement activities (if needed)
    * Enqueue new usage page */
   enlist_pgn_node(&caller->mm->fifo_pgn, pgn + pgit);
   return 0;
 }
```

Giải thích: Hàm vmap_page_range có vai trò ánh xạ (map) một dải các trang ảo của tiến trình vào các frame vật lý tương ứng trong bộ nhớ RAM. Khi tiến hành ánh xạ, hàm này sẽ lấy

địa chỉ bắt đầu (addr) và số lượng trang (pgnum), sau đó lần lượt gán frame vật lý vào bảng trang (page table) của tiến trình (caller->mm->pgd) tại các vị trí tương ứng. Đặc biệt, nó thiết lập các bit cho mỗi entry trong bảng trang để đánh dấu rằng trang hiện đang hiện diện trong RAM (bit present = 1). Đồng thời, hàm cũng cập nhật thông tin của vùng bộ nhớ đã ánh xạ vào ret_rg (vùng đã cấp phát) và ghi nhận các số trang được ánh xạ vào danh sách FIFO (fifo_pgn) nhằm phục vụ cho các thuật toán quản lý thay trang sau này (ví dụ như thay trang theo thứ tự vào/ra). Nhờ có vmap_page_range, tiến trình có thể truy cập từ địa chỉ ảo đến đúng địa chỉ vật lý trong RAM một cách chính xác.

2.4.2 init_mm(...)

```c
int init_mm(struct mm_struct *mm, struct pcb_t *caller)
{
  struct vm_area_struct *vma0 = malloc(sizeof(struct vm_area_struct));
  mm->pgd = malloc(PAGING_MAX_PGN * sizeof(uint32_t));
  /* By default the owner comes with at least one vma */
  vma0->vm_id = 0;
  vma0->vm_start = 0;
  vma0->vm_end = vma0->vm_start;
  vma0->sbrk = vma0->vm_start;
  struct vm_rg_struct *first_rg = init_vm_rg(vma0->vm_start, vma0->vm_end);
  enlist_vm_rg_node(&vma0->vm_freerg_list, first_rg);
  /* TODO update VMA0 next */
  vma0->vm_next = NULL;
  /* Point vma owner backward */
  vma0->vm_mm = mm;
  /* TODO: update mmap */
  mm->mmap = vma0;
  return 0;
}
```

Giải thích: Hàm init_mm đóng vai trò khởi tạo bộ quản lý bộ nhớ phân trang cho một tiến trình mới. Khi một tiến trình được tạo ra, hệ thống cần thiết lập trước không gian bộ nhớ ảo cho nó, và init_mm làm công việc đó. Cụ thể, hàm sẽ cấp phát bảng trang chính (page directory — pgd) để lưu trữ các entry ánh xạ trang ảo → frame vật lý. Đồng thời, nó cũng khởi tạo một vùng bộ nhớ ảo ban đầu (vm_area_struct), thiết lập các tham số như điểm bắt đầu vm_start, điểm kết thúc vm_end (cả hai đều là 0 lúc ban đầu) và sbrk — con trỏ dùng để quản lý heap. Bên cạnh đó, init_mm còn khởi tạo một danh sách vùng nhớ trống (free region list) để sau này

cấp phát heap thuận tiện. Hàm này liên kết vùng nhớ (VMA) với tiến trình (pcb_t) thông qua mm_struct, đặt nền móng cho mọi hoạt động cấp phát, giải phóng hay ánh xạ bộ nhớ tiếp theo của tiến trình trong hệ điều hành mô phỏng.

2.5 memphy

memphy là mô-đun mô phỏng bộ nhớ vật lý dưới dạng mảng byte (storage[]) với cơ chế quản lý các khung trang qua danh sách liên kết. Khi tiến trình yêu cầu cấp phát, hệ thống cấp phát frame từ free_fp_list, gán vào bảng trang. Các hàm MEMPHY_read, MEMPHY_write cho phép truy cập nội dung của bộ nhớ vật lý tương tự như RAM thật sự. Một số chức năng của các hàm:

Khởi tạo bộ nhớ vật lý – init_memphy & MEMPHY_format

Cấp phát / Giải phóng frame – MEMPHY_get_freefp, MEMPHY_put_freefp

Truy xuất bộ nhớ – MEMPHY_read, MEMPHY_write, MEMPHY_seq_*

Hoán trang – __swap_cp_page(...)

2.5.1 MEMPHY_dump(...)

```c
int MEMPHY_dump(struct memphy_struct *mp)
 {
   /*TODO dump memphy contnt mp->storage
    *     for tracing the memory content
    */
   if (mp == NULL || mp->storage == NULL) return -1;
   printf("===== PHYSICAL MEMORY DUMP =====\n");
   for (int i = 0; i < mp->maxsz; i++) {
     if (mp->storage[i] != 0) {
       printf("BYTE %08x: %d\n", i, mp->storage[i]);
     }
   }
   printf("===== PHYSICAL MEMORY END-DUMP =====\n");
   printf("================================\n");
   return 0;
 }
```

Giải thích: Hàm MEMPHY_dump nhằm mục đích hiển thị toàn bộ nội dung của bộ nhớ vật lý (memphy) để phục vụ cho việc kiểm tra (debug) và theo dõi trạng thái bộ nhớ. Hàm này

sẽ duyệt qua tất cả các ô nhớ (mảng storage) trong thiết bị bộ nhớ vật lý (cấu trúc struct memphy_struct) và in ra những ô nhớ nào đang được sử dụng (tức là các byte có giá trị khác 0).

2.6 Testcase os_1_mlq_paging

Input:

```
2 4 8
1048576 16777216 0 0 0
1 p0s  130
2 s3  39
4 m1s  15
6 s2  120
7 m0s  120
9 p1s  15
11 s0 38
16 s1 0
```

p0s	s3	m1s	s2	m0s	p1s	s0	s1
1 14	7 11	1 6	20 12	1 6	1 10	12 15	20 7
calc	calc	alloc 300 0	calc	alloc 300 0	calc	calc	calc
alloc 300 0	calc	alloc 100 1	calc	alloc 100 1	calc	calc	calc
alloc 300 4	calc	free 0	calc	free 0	calc	calc	calc
free 0	calc	alloc 100 2	calc	alloc 100 2	calc	calc	calc
alloc 100 1	calc	free 2	calc	write 102 1 20	calc	calc	calc
write 100 1 20	calc	free 1	calc	write 1 2 1000	calc	calc	calc
read 1 20 20	calc		calc		calc	calc	calc
write 102 2 20	calc		calc		calc	calc	
read 2 20 20	calc		calc		calc	calc	
write 103 3 20	calc		calc		calc	calc	
read 3 20 20	calc		calc		calc	calc	
calc			calc			calc	
free 4						calc	
calc						calc	
						calc	

Hình 3: input

Output:

```
Time slot  24
        CPU 2: Put process  7 to run queue
        CPU 2: Dispatched process  7
```

```
        CPU 3: Processed  1 has finished
        CPU 3 stopped
Time slot  25
Time slot  26
        CPU 2: Put process  7 to run queue
        CPU 2: Dispatched process  7
Time slot  27
        CPU 2: Processed  7 has finished
        CPU 2 stopped
```

Time slot 0: Tiến trình p0s được nạp vào hệ thống, Gán PID = 1, Priority = 130. CPU 3 bắt đầu thực thi tiến trình này.

Time slot 2: PID 1 thực hiện ALLOC 300 byte tại Region 0 → bắt đầu từ địa chỉ 0x00000000. Tiến trình s3 được nạp, PID = 2, Priority = 39 (cao hơn PID 1). CPU 3 chuyển PID 1 về hàng đợi. CPU 3 chạy PID 2, CPU 2 chạy lại PID 1.

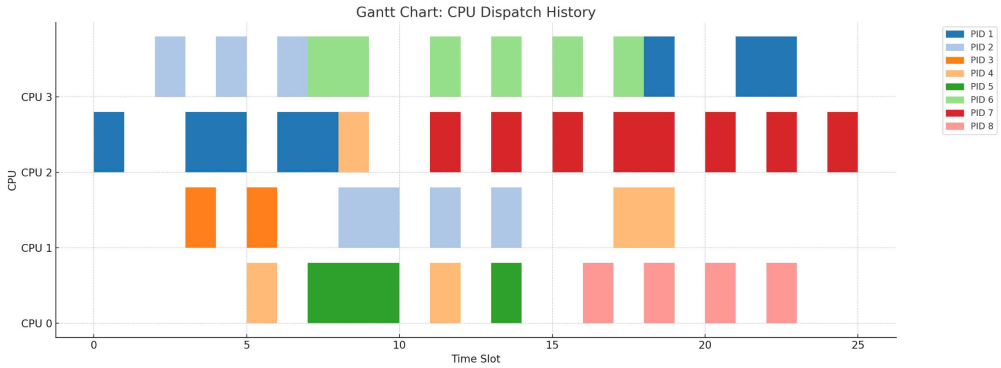
Time slot 3: PID 1 thực hiện ALLOC 300 byte cho Region 4 bắt đầu tại 0x00000200. Tiến trình m1s được nạp vào với Priority 15 → ưu tiên cao.

Time slot 4: CPU 0 chạy PID 3 (m1s), CPU 2 chạy lại PID 1, CPU 3 chạy lại PID 2. Time slot 5: ghi giá trị 100 vào địa chỉ tương ứng region 1 + 20.

....

Các tiến trình luân phiên chạy giữa các CPU. Time slot 23–27: các tiến trình lần lượt kết thúc.

Dưới đây là Gantt chart:



Hình 4: Gantt chart input os 1 mlq paging

Một số mốc thời gian quan trọng trong time slot của input trên

Time Slot	Sự kiện nổi bật
0	Bắt đầu mô phỏng, load PID 1
1–2	Cấp phát vùng nhớ đầu tiên, load PID 2
3	Load PID 3, xuất hiện phân trang lớn
5	Load PID 4, bắt đầu nhiều CPU phối hợp
7	`read`, `write`, dump RAM
9	Load PID 6, PID 3 kết thúc
13–14	`write`, ghi page và RAM
16	Load PID 8 (cuối cùng)
21–26	Các tiến trình kết thúc, CPU shutdown

3 System Call

3.1 Trả lời câu hỏi

Question: What is the mechanism to pass a complex argument to a system call using the limited registers?

- Để truyền một đối số phức tạp cho lệnh gọi hệ thống sử dụng thanh ghi giới hạn, có thể sử dụng cơ chế **truyền địa chỉ vùng nhớ** (con trỏ) chứa dữ liệu thông qua thanh ghi.
- Ví dụ, địa chỉ của tham số được lưu trong thanh ghi, và kernel sẽ đọc dữ liệu từ vùng nhớ tương ứng. Trong system call killall, RIGIONID là một ví dụ, nơi ID vùng nhớ tham chiếu đến dữ liệu phức tạp (tên process) được lưu trữ trong memory.

Question: What happens if the syscall job implementation takes too long execution time?

- Nếu system call thực thi quá lâu sẽ gây ra hậu quả là treo hệ thống hoặc giảm hiệu suất do kernel bị chiếm dụng. Đồng thời, các tiến trình khác không được lập lịch dẫn tới starvation.

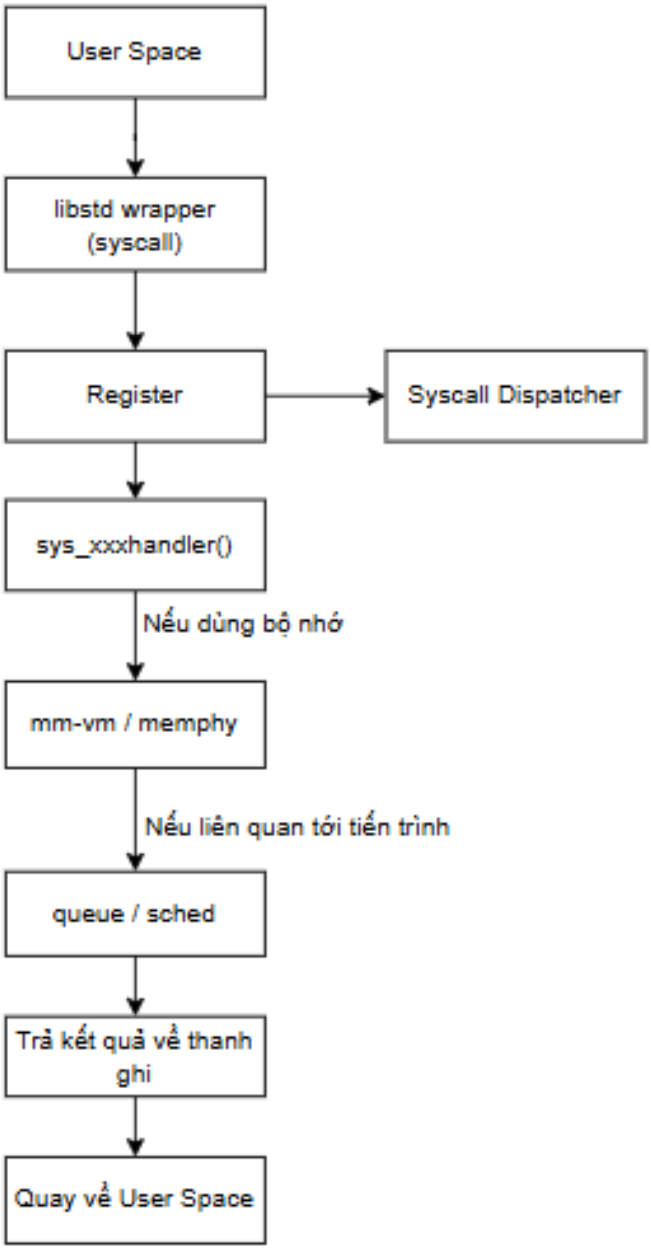
Question: What happens if the synchronization is not handled in your Simple OS? Illustrate the problem of your simple OS (assignment outputs) by example if you have any.

Trong hệ điều hành Simple OS, nếu không dùng lock/mutex để bảo vệ tài nguyên chia sẻ (như ready_queue, memory frames, page tables), sẽ xảy ra:

- Race Condition: Nhiều CPU cùng truy cập/sửa đổi một tài nguyên, dẫn đến dữ liệu bị ghi đè hoặc hỏng.

- Inconsistent State: Trạng thái hệ thống không nhất quán (vd: một process bị đưa vào nhiều queue).
- Deadlock/Starvation: Process bị treo do tranh chấp tài nguyên.

3.2 Sự tương tác giữa các module khi gọi System Call



Khi tiến trình trong User space gọi một System call, như:

```
syscall(404,1);
```

Dựa vào số hiệu syscall, hệ thống kernel tra cứu bảng syscall.tbl để xác định hàm xử lí tương ứng như:

```
0       listsyscall sys_listsyscall
17      memmap      sys_memmap
101     killall     sys_killall
440     xxx         sys_xxxhandler
```

Sau đó, kernel gọi đúng hàm xử lí:

```
int __sys_xxxhandler(struct pcb_t *caller, struct sc_regs *regs)
```

Tại đây, logic chính được xử lí, ví dụ:

```
printf("The first system call parameter %d\n", regs->a1);
```

Các module phụ trợ tương tác:

Module	Vai trò khi được `sys_xxxhandler` gọi
`mm-vm.c`	Dịch địa chỉ ảo sang địa chỉ vật lý, xử lý phân vùng
`paging.c`	Quản lý bảng trang, cấp phát khung trang
`mm-memphy.c`	Đọc/ghi dữ liệu tại RAM hoặc SWAP
`queue.c`	Truy xuất hàng đợi ready/running
`sched.c`	Thay đổi trạng thái tiến trình

Cuối cùng là trả về kết quả chương trình người dùng.

3.3 Triển khai System Call killall

System call killall có nhiệm vụ dừng tất cả các tiến trình có tên trùng khớp với tên được chỉ định. Báo cáo này trình bày chi tiết các yêu cầu cần thiết để triển khai chức năng này, tập trung vào:

1. Lấy tên tiến trình từ bộ nhớ.
2. Tìm và dừng các tiến trình phù hợp.
3. Tương tác giữa các module hệ thống.

3.3.1 Lấy tên tiến trình từ bộ nhớ

```
int i = 0;
data = 0;
while(data != -1){
    libread(caller, memrg, i, &data);
    proc_name[i]= data;
    if(data == -1 || i >= 99) proc_name[i]='\0';
    i++;
}
printf("The procname retrieved from memregionid %d is \"%s\"\n", memrg
    , proc_name);
```

Mục đích: Đọc chuỗi tên tiến trình (kiểu char*) được lưu trong một vùng nhớ ảo của tiến trình gọi syscall, với chỉ số vùng nhớ được truyền qua thanh ghi regs→a1.

Giải thích: Trong vòng lặp while gọi hàm libread() để đọc 1 byte tại offset i từ vùng nhớ có region_id = memrg. Ghi giá trị vừa đọc vào proc_name[i]. Nếu gặp kí tự kết thúc chuỗi '\0' hoặc đạt giới hạn mảng thì gán kết thúc chuỗi rồi thoát.

3.3.2 Duyệt running_list: Kết thúc tiến trình đang thực thi

```
if (caller->running_list && caller->running_list->size > 0) {
    struct pcb_t **procs = caller->running_list->proc;
    int new_size = 0;
    for (int i = 0; i < caller->running_list->size; i++) {
        if (!procs[i]) continue;
        if (strcmp(procs[i]->path, proc_name) != 0) {
            procs[new_size++] = procs[i]; //  G i    l i
        } else {
            printf("Terminating running process %d (%s)\n",
                procs[i]->pid, procs[i]->path);
            free(procs[i]->mm);
            free(procs[i]);
            procs[i] = NULL;
        }
    caller->running_list->size = new_size;
    }
}
```

Mục tiêu:

- Tìm các tiến trình đang thực thi (do tiến trình hiện tại gọi syscall quản lý).
- Nếu chương trình của tiến trình khớp với proc_name, giải phóng bộ nhớ và xoá khỏi danh sách.

Giải thích: Kiểm tra nếu danh sách tiến trình đang chạy tồn tại và không rỗng. Sau đó, tạo con trỏ đến danh sách tiến trình và biến đếm mới để cập nhật lại danh sách. Dùng vòng for để duyệt từng tiến trình trong danh sách hiện tại, nếu phần tử proc[i] rỗng thì bỏ qua. Nếu không trùng tên giữ lại tiến trình sau đó copy vào vị trí mới (new_size). Nếu trùng tên, giải phóng bộ nhớ ảo và xoá PCB, không đưa vào danh sách mới. Cuối cùng, cập nhật lại kích thước danh sách → loại bỏ hoàn toàn tiến trình đã xoá.

3.3.3 Duyệt ready_queue: Kết thúc tiến trình đang chờ

```c
if(caller->ready_queue && caller->ready_queue->size > 0) {
    struct queue_t *q = caller->ready_queue;
    for (int i = 0; i < MAX_PRIO; i++) {
        struct pcb_t **procs = q[i].proc;
        int new_size = 0;
        for (int j = 0; j < q[i].size; j++) {
            if (!procs[j]) continue;
            if (strcmp(procs[j]->path, proc_name) != 0) {
                procs[new_size++] = procs[j];
            } else {
                printf("Terminating ready process %d (%s)\n",
                    procs[j]->pid, procs[j]->path);
                free(procs[j]->mm);
                free(procs[j]);
                procs[j] = NULL;
            }
        }
        q[i].size = new_size;
    }
}
```

Mục tiêu: Tìm các tiến trình đang chờ. Nếu chương trình của tiến trình khớp với proc_name, giải phóng bộ nhớ và xoá khỏi danh sách.

Giải thích: Tương tự như duyệt running_list. Duyệt tất cả hàng đợi theo mức ưu tiên (MAX_PRIO), mỗi hàng đợi chứa danh sách tiến trình có cùng mức độ ưu tiên, Với mỗi hàng đợi tạo một danh sách mới rút gọn bằng cách giữ lại các tiến trình không bị kill. Nếu tiến trình có tên khợp với

proc_name thì thực hiện giải phóng bộ nhớ ảo và giải phóng tiến trình. Sau cùng, cập nhật lại kích thước hàng đợi để loại bỏ các tiến trình đã xóa.

3.3.4 Sự tương tác giữa các module khi gọi killall

System call `killall` là một ví dụ tiêu biểu thể hiện sự phối hợp giữa nhiều module trong hệ điều hành đơn giản, bao gồm: quản lý bộ nhớ, điều khiển tiến trình và quản lý hàng đợi lập lịch. Dưới đây là mô tả chi tiết luồng tương tác giữa các module khi lời gọi `killall` được thực thi:

- `libstd` – **Giao diện người dùng:**

  Tiến trình người dùng gọi: `syscall(101, REGIONID);`

  Trong đó, `REGIONID` là chỉ số vùng nhớ chứa tên chương trình cần dừng. Lời gọi này được chuyển vào kernel thông qua syscall số 101.

- `__sys_killall` – **Xử lý system call trong kernel:**

  Hàm `__sys_killall`:

  - Gọi `libread()` để đọc chuỗi tên từ vùng nhớ ảo (dựa trên `REGIONID`).
  - Dữ liệu sẽ được lấy từng byte nhờ các hàm hỗ trợ đọc bộ nhớ ảo.

- `mm-vm.c`, `mm.c`, `paging.c` – **Bộ quản lý bộ nhớ ảo:**

  Các module này chịu trách nhiệm:

  - Tra địa chỉ ảo để xác định trang tương ứng.
  - Nếu chưa được ánh xạ → dùng `SYSCALL_MEMINC` để cấp phát.
  - Nếu trang nằm ở SWAP → dùng `SYSCALL_MEMSWP` để chuyển vào RAM.

  Chuỗi tên chương trình được đọc byte-by-byte thông qua hệ thống phân trang.

- `pcb_t`, `queue.c`, `sched.c` – **Quản lý tiến trình và hàng đợi:**
  Sau khi đã có `proc_name`, handler:

  - Duyệt danh sách `running_list`: Nếu khớp tên, giải phóng bộ nhớ và xoá khỏi danh sách.
  - Duyệt tất cả hàng đợi `ready_queue[i]`: Nếu khớp tên → xoá hoàn toàn khỏi hàng đợi.

3.3.5 Testcase os_syscall

Input: 2 CPU, 1 tiến trình đầu vào, 1 syscall trong tiến trình . Tại time slot 9 nạp tiến trình từ input/proc/sc2 với ưu tiên 15 (cao).

```
2 1 1
2048 16777216 0 0 0
9 sc2 15
```

Với thời gian chạy tối đa là 20 time units, có 5 lệnh được thực thi bao gồm:

Lệnh	Giải thích
`alloc 100 1`	Cấp phát 100 bytes vào vùng nhớ ảo với `region_id = 1`. Đây là nơi dùng để ghi tên chương trình cần dừng trong system call `killall`.
`write 80 1 0`	Ghi byte `80` vào offset `0` của `region_id = 1`. `80` là mã ASCII của ký tự `'P'`.
`write 48 1 1`	Ghi byte `48` vào offset `1`. `48` là mã ASCII của ký tự `'0'`.
`write -1 1 2`	Ghi giá trị `-1` vào offset `2` để đánh dấu kết thúc chuỗi. `-1` được dùng thay cho ký tự `'\0'` để mô phỏng kết thúc chuỗi.
`syscall 101 1`	Gọi system call `killall` với tham số `REGIONID = 1`. Kernel sẽ: • Đọc chuỗi `"P0"` từ vùng nhớ ảo của tiến trình. • Duyệt danh sách các tiến trình hiện tại. • Nếu tiến trình nào có `pcb->path == "P0"` thì sẽ bị xoá hoàn toàn.

```
20 5
alloc 100 1
write 80 1 0
write 48 1 1
write -1 1 2
syscall 101 1
```

Output:

```
Time slot   0
ld_routine
Time slot   1
Time slot   2
Time slot   3
Time slot   4
```

```
Time slot   5
Time slot   6
Time slot   7
Time slot   8
Time slot   9
    Loaded a process at input/proc/sc2, PID: 1 PRIO: 15
Time slot  10
    CPU 0: Dispatched process  1
Time slot  11
Time slot  12
    CPU 0: Put process  1 to run queue
    CPU 0: Dispatched process  1
Time slot  13
Time slot  14
    CPU 0: Put process  1 to run queue
    CPU 0: Dispatched process  1
The procname retrieved from memregionid 1 is "P0"
Time slot  15
    CPU 0: Processed  1 has finished
    CPU 0 stopped
```

Giải thích:

Time Slot	Diễn giải
$0 \to 8$	Không có gì xảy ra. Chờ đến thời điểm nạp tiến trình.
9	Tiến trình `sc2` được nạp vào bộ nhớ, gán `PID = 1`, `PRIO = 15`.
10	CPU 0 dispatch tiến trình 1 → tiến trình bắt đầu chạy.
12	CPU đưa tiến trình 1 vào run queue, rồi lại dispatch để tiếp tục chạy.
14	Tương tự: tiến trình 1 quay lại run queue và được dispatch lần nữa.
(giữa 14–15)	Hàm `__sys_killall` được gọi → bắt đầu xử lý syscall. `The procname retrieved from memregionid 1 is "P0"` Trong tiến trình `sc2`, gọi `killall` với `REGIONID = 1`. Vùng nhớ này chứa chuỗi `"P0"` – tên chương trình cần kill.
15	CPU kết thúc tiến trình 1 (`process 1 has finished`) → tức là `sc2` đã chạy xong sau khi gọi syscall.